

Dark Impact

1. Project Overview

Dark Impact is a 2D side-scrolling action game built with Pygame, and running on a fixed resolution of 1280×720. The player controls a knight warrior battling through five stages, each with its own difficulty and rewards. There are four common enemy types, which are Skeleton, Mushroom, Goblin, and Flying Eye and three random bosses.

Mechanics: Players can freely choose any stage, with harder stages offering greater rewards. Core actions include running, jumping, dashing, and combat attack animations. Defeating enemies earns points used to upgrade the character between runs.

Functionality: The game flows through a main menu, level selection, and individual stages. Each stage handles input, enemy AI, collision detection, and HUD rendering. Results are logged after each session for statistical analysis.

2. Project Review

Dead Cells, a 2D side scrolling action platformer where the player fights through procedurally generated stages, defeats various enemies and bosses, and earns cells to upgrade their character between runs.

Planned improvements over Dead Cells:

- **Free stage selection:** *Dead Cells* forces linear progression; *Dark Impact* lets players choose any stage with rewards scaled to difficulty
- **Statistical tracking:** *Dark Impact* records win count, enemies defeated, stage completion, and session data for analysis, which *Dead Cells* does not offer to the player
- **Focused design:** rather than procedural generation, *Dark Impact* uses hand-crafted stages and a focused roster of four enemy types

and three bosses, making gameplay more consistent and predictable for the player

3. Programming Development

3.1 Game Concept

Dark Impact is a 2D side scrolling action game where the player controls a knight warrior fighting through five stages. The objective is to defeat all enemies in a stage, survive against bosses, and earn points to strengthen the character for harder challenges.

Objectives:

- *Defeat all common enemies and the stage boss to complete a stage*
- *Earn points based on stage difficulty and enemies defeated*
- *Spend points to upgrade character stats between runs*
- *Track personal performance through recorded statistics*

Mechanics:

- **Movement:** *run, jump, and dash*
- **Combat:** *attack using three attack types (attack1, attack2, combo) with hitbox-based collision*
- **Enemies:** *four common types (Skeleton, Mushroom, Goblin, Flying Eye) each with patrol and attack AI*
- **Bosses:** *three random bosses with unique attack patterns, one per selected stage*
- **Upgrades:** *points earned can be spent on speed, attack power, and health improvements*
- **Stage selection:** *all five stages are freely accessible; harder stages give greater rewards*

Key Selling Points:

- *Fast and responsive combat with multiple attack animations*
- *Free stage selection with difficulty-based reward scaling*

- *Light progression system that rewards repeated play*
- *Statistical tracking, win count, enemies defeated, and completion data saved each session*

3.2 Object-Oriented Programming Implementation

1. System *Central wrapper around the pygame display. Passed to all game screens to handle rendering and screen info.*

Attributes:

- *w, h, screen*

Methods:

- *get_screen(), setBg(), draw_rect(), draw_circle(), add_text(), paragraph(), get_resolution(), initialize(), stop()*

2. Player *represents the player character. Handles movement, physics, animation state, and directional logic, it will inherit attributes such as health, attack, etc.. from Entity.*

Attributes:

- *rect, surf, velo, on_ground, is_jumping, is_dashing, is_facing_left/right, jump_cooldown, frames, frame_index*

Methods:

- *update(), jump(), dash(), set_state(), face_left(), face_right()*

3. Knight *Sprite loader for player characters. Loads sprite sheets and slices them into frames per animation state.*

Attributes:

- *sprite_set, knight_state*

Methods:

- *load_sprite(), get_sprite(), get_sprite_set(), add_image()*

4. Goblin / FlyingEye / Mushroom / Skeleton / (and bosses) *Sprite loader for enemy characters. Loads sprite sheets and slices them into frames per animation state.*

Attributes:

- *sprite_set, enemy_state*

Methods:

- *load_sprite(), get_sprite(), get_sprite_set(), add_image()*

5. Entity *(planned base class) Parent class shared by all enemies and player. Contains common combat attributes that all enemy types inherit.*

Attributes:

- *hp, attack_power, speed, is_alive, rect, surf*

Methods:

- *take_damage(), die(), update()*

6. Enemy *(Base class for Boss and enemies) Handles enemy, detection, and attacking. Each type (Skeleton, Mushroom, Goblin, Flying Eye) extends this with unique behavior.*

Attributes:

- *state, hitbox*

Methods:

- *detect_player(), attack(), animate()*

7. Button *Reusable UI button used across all menu screens. Renders text with hover effects and handles click detection.*

Attributes:

- *rect, text, font, font_size, surface*

Methods:

- *create(), is_clicked()*

8. Level *Manages a single playable stage. Loads the background, initializes the player, and runs the game loop.*

Attributes:

- *sys, screen, current_lv*

Methods:

- *show()*

9. (Skeleton, Goblin, Mushroom, BigMushroom, FlyingEye, Minotaur, Golem, TarnishedWidow) *Inherent from Enemy class with different behaviors*

Attributes:

- *State, hitbox, spawn_x*

Methods:

- *Update, attack, is_dead*

3.3 Algorithms Involved

1. Event-Driven Input Handling Player actions such as jump and dash are triggered by single keydown events, while movement uses per-frame key polling. This separates instant actions from continuous ones.

2. Enemy AI Walk Toward Player Each enemy checks the player's x position every frame and moves in that direction. When within attack range, it switches to attack state.

3. Dodge / Invincibility Frame Logic When the player is in a rolling/dash state, collision with an enemy attack is ignored. The game checks `is_dashing` before applying damage, if true, the hit is cancelled.

4. Spawn Management The level initializes a fixed enemy limit. Enemies spawn one at a time from a random side of the screen and walk toward the player. When all are defeated, the boss spawns at a fixed position.

5. Hitbox Collision Detection `pygame.Rect.colliderect()` checks overlap between the player's attack box and enemy hitbox each frame to register hits. The same check is used for enemy attacks against the player's hitbox.

4. Statistical Data (Prop Stats)

4.1 Data Features

1. Type of Enemies Defeated *Number of enemies defeated per session (accumulative), broken down by enemy type (Skeleton, Mushroom, Goblin, Flying Eye) and boss kills.*

2. Health *Collect player health by each section (time stamp accumulative)*

3. Points *Collect player points by each section (time stamp accumulative)*

4. Player choice of upgrade *Collect user data of upgrading choices.*

5. Player strength *Player strength (health, stamina, stamina regeneration,) basically a level tracking procedure.*

	Why it's good to have this data? what can it be used for	How to obtain 100 values of this feature data	What variable and class will you collect this from	How will you display this feature data
During stage points tracking	To track user point earned from stage enemies (accumulative)	Time stamp every 5-10 seconds	Variable: self.point_earned Class: Level	Line chart
During stage health tracking	To track damage dealt onstage (accumulative)	Time stamp every 5-10 seconds	Variable: self.health Class: Player	Line chart
Upgrade points usage tracking	To track player point usage while upgrading (imagine having 100 points, which we can only use 30 and left 70 to use in the future)	Everytime player click upgrade and successful	Variable: player['accumulative_points'] Class: No class, this is public variable from stat.py file	Pie chart
Enemy type defeated	To track type of enemy defeated (accumulative)	Kill 100 enemies	Variable: self.enemy_killed Or self.enemy_type_killed Class: Level	Bar chart
Player strength	To track player strength when upgrading overtime	Everytime player click upgrade and successful (need to have sufficient point) 100 times	Variable: There will be based public dict named player which store every play stats, but the state is received from the latest row of the player stat table Class: PlayerStat	Pie chart

4.2 Data Recording Method

The game will store statistical data in sqlite3 .db file. After each session ends whether the stage is completed or the player dies, the game saves a new row to the histories.db containing all tracked features for that session.

Also the player status (strength) will be record in different table, but in the same [histories.db](#) file

4.3 Data Analysis Report

I will use python available library to analyze the recorded data such as pandas or others.

Statistical Measures:

- **Barchart, Piechart, Table, Line Chart**

Presentation:

- **Bar chart:** enemies defeated per type across all sessions
- **Line graph:** player score and health overtime (timestamp during each game section)
- **Pie chart:** Player choice of upgrade
- **Table:** summary of each session showing mean, median, stdev of health, points, time (every game session combined)

5. Project Planning Timeline

Week	Week	Task
1	8	Proposal submission / Project initiation
2	9	Implement UIs (menu, level selection, stage, buttons, label, and ui layout)
3	10	Complete player movement, complete slicing sprites for all, implement sprites to player and other enemies.

4	11	Implement attack, hitboxes, roll/dodge, health, stamina and add behavior for all enemies.
5	12	Continue from week 11 if not finished, implement level completion and reward, statistical implementation.
6	13	Bug fix, user test, add other functions if available.
7	14	Submission week (Draft)

6. Document version

Version: 1.0

Date: 13 March 2025

Editor: Pasin Makcharoen

Date	Name	Description of Revision, Feedback, Comments
14/3	Nattanan	There is the sprite management feature in pygame already Checkout: https://www.pygame.org/docs/ref/sprite.html So some classes for sprite management might be unnecessary. Managing it inside the character class as a whole is better. (in my opinion) Also if enemy have different behavior, try using inheritance. The data presentation still not right, study the data science first then revisit this later

	Feature Name	Graph objective	Graph type	X-axis	Y-axis
Graph 1-1	During stage points tracking	To show player accumulative point progression from first play to latest play And summarize	Line chart	Time	Amount
Graph 1-2	During stage health tracking	To show player health progression from first play to latest play And summarize	Line chart	Time	Amount
Graph 2	Upgrade points usage tracking	Display point usage by player	Pie chart	- (parameters - Health - Power - Critical - Stamina - Stamina - Regen Speed)	

Graph 3	Enemy type defeated	Display every enemy defeated with species parameter	Bar chart	Species ex. (skeleton, goblin, etc..)	Number defeated
Table 1	Overall summarizations	Summarize user health, points, and time playing the game	Table	Health, points, time	Mean, median, stdev